



version 1.0

User manual

Mico Mrkaic

May 29, 2026

Abstract. `tea` is a small command-line program for the data-prep-to-quick-estimate loop economists use Stata for: import, clean, generate variables, run regressions, panel methods, MLE, IV, and ARIMA. The syntax is close to Stata's where they overlap; the semantics are faithful where it matters (missing-value algebra, the `by:` prefix, gap-aware time-series operators, factor variables). This manual covers installation, a quickstart tour, all v1.0 commands and estimators, known limitations, and escape hatches to Python/R for things `tea` does not handle.

Audience. Applied economists, policy analysts, and graduate students familiar with Stata. No Stata experience is strictly required, but the manual assumes you know what `regress` is and what panel data looks like.

Status. v1.0 is the first frozen release. It has passed 39 regression tests, is clean under AddressSanitizer and UndefinedBehaviorSanitizer, and builds warning-free under `-Wall -Wextra -Werror`. It has *not* yet been tested in the field — that's what this version is for. Please report what breaks.

AI-assisted development. This software was developed with the assistance of an AI coding assistant (Anthropic's Claude). The design decisions, mathematical specifications, scope choices, and final acceptance of each component were the author's; the AI contributed substantial portions of the implementation, drafted regression tests, and produced the first draft of this manual. All code was reviewed before inclusion. This manual was likewise drafted with AI assistance and edited by the author. Bugs and design flaws are the author's responsibility.

License. GPLv3 or later. Copyright © 2026 Mico Mrkaic. See the `LICENSE` file in the distribution for the full text. `tea` is an independent open-source project, not affiliated with or endorsed by StataCorp LLC. *Stata* is a registered trademark of StataCorp LLC.

Contents

1	Introduction	6
1.1	What <code>tea</code> is	6
1.2	What <code>tea</code> is not	6
1.3	Design choices, locked	6
2	Installation	6
2.1	Build from source	6
2.2	Dependencies	7
2.3	Sanity check	7
2.4	Running interactively	7
3	Your first <code>tea</code> session	8
4	The <code>do</code>-file language	8
4.1	Comments	9
4.2	Macros	9
4.3	Control flow	9
4.4	Capture and quietly	10
4.5	The <code>by:</code> prefix	10
5	Data management	10
5.1	Importing data	10
5.2	Saving data	11
5.3	Generating and modifying variables	11
5.4	<code>egen</code> extended generators	11
5.5	Type conversion	11
5.6	Labels	12
6	Exploring data	12
6.1	<code>describe</code>	12
6.2	<code>list</code>	12
6.3	<code>summarize</code>	12
6.4	<code>tabulate</code>	12
6.5	<code>codebook</code>	13
6.6	<code>count</code>	13
7	Reshaping, merging, aggregating	13
7.1	<code>sort</code> and <code>gsort</code>	13
7.2	<code>reshape</code>	13
7.3	<code>merge</code>	13
7.4	<code>collapse</code>	14
7.5	Frames	14
8	Time series and panel data	14
8.1	Declaring panel / time structure	14
8.2	<code>xtdescribe</code>	15

9	Factor variables	15
9.1	Atoms	15
9.2	Interactions	15
9.3	Coefficient naming	15
9.4	Known limitation: factor $\times$ TS-op composition	16
10	Linear regression	16
10.1	Basic syntax	16
10.2	Output	16
10.3	Robust and clustered SEs	16
10.4	Constraints and hypothesis tests	16
11	Panel regression	17
11.1	Fixed effects	17
11.2	Random effects	17
11.3	Between effects	17
11.4	Hausman test	17
12	Maximum likelihood estimators	17
12.1	logit	18
12.2	probit	18
12.3	poisson	18
12.4	Marginal effects via margins	18
13	Instrumental variables	18
13.1	Syntax	18
13.2	Example	19
13.3	Interpreting the first-stage F	19
13.4	Variance estimators	19
14	Time series models	19
14.1	ARIMA	19
14.2	Example	19
15	Post-estimation	20
15.1	predict	20
15.2	Hypothesis tests	20
16	Output and reproducibility	20
16.1	Saving estimates	20
16.2	estout: publication tables	21
16.3	log	21
17	Random numbers and Monte Carlo	22
17.1	Reproducibility	22
17.2	Functions	22
17.3	A Monte Carlo example	22
18	Escape hatches: when to leave tea	22

18.1	When to escape	23
18.2	A bridge example	23
19	Known v1.0 limitations	23
19.1	Data and metadata	23
19.2	Econometric estimators	23
19.3	Factor variables	24
19.4	Programming	24
19.5	Graphics	24
20	Reporting bugs	24
A	Command reference	25
B	Function reference	28
B.1	Arithmetic and rounding	28
B.2	Exponential and logarithm	28
B.3	Trigonometric and hyperbolic	28
B.4	Probability distributions	29
B.5	Random number generation	29
B.6	Missing-value tests and ranges	29
B.7	String construction and inspection	29
B.8	String transformation	30
B.9	Regular expressions (POSIX extended)	30
B.10	Numeric ↔ string conversion	30
B.11	Calendar: building dates	30
B.12	Calendar: extracting components from a daily date	31
B.13	Calendar: converting between frequencies	31
B.14	Special: running and time-series-context	31
C	egen function reference	31
C.1	Group-wise aggregators	31
C.2	Position and identity	32
C.3	Row-wise aggregators (across columns of a single row)	32
D	Summarize / tabstat statistics keywords	32
E	Stata cheat sheet	33

1. Introduction

1.1 What tea is

tea is a C17 program that runs as either an interactive REPL or a batch processor for do-files. It does the data manipulation and econometrics that most applied economists do most days — and nothing else. No plotting, no Bayesian inference, no machine learning. For those, escape to Python or R; see §18.

The design priority is *Stata fidelity in the overlapping subset*. A do-file that runs in **tea** should run in Stata with the same results, modulo a handful of small documented differences. Where Stata's behavior includes a footgun (if `x > 5` includes missing values, for instance, because missing sorts above every number), **tea** keeps the footgun. That choice is deliberate: it makes **tea** a drop-in for the data-prep phase, even if you eventually move to Stata or R for the final analysis.

1.2 What tea is not

tea v1.0 does *not* implement: graphics of any kind, exact-ML ARIMA (we use conditional likelihood), seasonal ARIMA, GARCH, mixed-effects models, survival analysis, structural breaks, VAR, VECM, cointegration tests, Bayesian inference, or machine learning. It also does not implement Stata's full programming language — **tea** has macros, loops, and **capture**, but no **program define**, no **syntax** parser, no Mata.

The **shell** command makes it easy to bridge to Python or R when you need any of the above. See §18.

1.3 Design choices, locked

- **Storage** is one of two types: **double** (numeric, with the full Stata missing-value algebra) or string. Stata's **byte**, **int**, **long**, **float** are cosmetic on read and compressed back on write of **.dta** files.
- **Multiple frames** are supported. Each has its own sort state and panel/time settings.
- **by:** is **strict**. It errors if the data isn't sorted; **bysort** sorts first.
- **reshape** is **in-place**. Use **frame copy** first if you want to keep the original.
- **m:m merges are rejected**. They are almost never what the user actually wants.
- **C17**, no novelties. Toolchain stability over newer-standard ergonomics.

2. Installation

2.1 Build from source

You will have received a tarball, **tea-v1.0.tar.gz**.

```
tar xzf tea-v1.0.tar.gz
cd tea
make
```

The build produces `./tea` in the project root. Move it somewhere on your `$PATH` (e.g. `~/bin`) or invoke directly with `./tea`.

2.2 Dependencies

You need a C17 compiler (`gcc 8+` or `clang 7+`) and the following libraries with their development headers:

Library	Why
<code>libreadline</code>	line editing, history, tab completion in the REPL
<code>libopenblas</code>	matrix kernels
<code>liblapacke</code>	LAPACK C interface (regression, ARIMA, IV)
<code>libgsl</code>	distributions for p -values and confidence intervals
<code>libreadstat</code>	Stata <code>.dta</code> (and SAS / SPSS) I/O

On Debian / Ubuntu:

```
apt install libreadline-dev libopenblas-dev liblapacke-dev \
libgsl-dev libreadstat-dev
```

On macOS (Homebrew):

```
brew install readline openblas lapack gsl readstat
```

Run `make check-deps` after installing to verify every required header is reachable. If any dependency is missing, that target reports which one and the install command to use.

2.3 Sanity check

```
./tea --version      # prints "tea 1.0 -- tiny econometric assistant"
./tea tests/demo.do  # batch: runs a short demo (no errors expected)
make test            # full regression suite, 39 tests
```

If `make test` reports `tea regression: 39/39 passed`, your build is correct.

2.4 Running interactively

Start the REPL by invoking `tea` with no arguments:

```
$ tea
tea 1.0 - tiny econometric assistant
type 'help' for commands, Ctrl-D to exit.
. _
```

History is persistent at `~/.tea_history` (capped at 2000 lines). Tab completes command names at the start of a line and variable names elsewhere. `help` lists commands, `help CMD` shows the syntax for one.

3. Your first tea session

Let's load a small panel dataset, look at it, and run a regression. Save the following as `quickstart.do`:

```
* quickstart.do
import delimited "tests/WEO.csv", clear
rename INDICATOR_ID indicator_id
keep if indicator_id == "NGDP_RPCH"
rename COUNTRY_ID country_id
keep country_id v*
reshape long v, i(country_id) j(year)
rename v growth
keep if !missing(growth)

* Restrict to G7 countries for tractable output
keep if inlist(country_id, "USA", "DEU", "GBR", "FRA", "ITA", "JPN", "CAN")

* Convert string country codes to numeric so we can use factor vars
encode country_id, gen(cid)
xtset cid year

summarize growth
xtdescribe

* Fixed-effects regression of growth on its own lag
xtreg growth L.growth, fe
```

Run it:

```
tea quickstart.do
```

This loads the IMF *World Economic Outlook* CSV (shipped with the distribution), pivots it from wide to long, restricts to G7 countries, sets up the panel structure with `xtset`, and runs a fixed-effects regression. Output looks like Stata's; if you've used Stata before, nothing here should surprise you.

A few things to notice:

- `rename`, `keep if`, `reshape long` all work the same as in Stata.
- `encode` maps string country IDs to consecutive integers 1, 2, 3, ... and attaches a value label so `list cid` still shows "USA", "DEU", etc.
- `xtset cid year` declares the panel structure. After this, `L.growth` means "growth lagged one year, gap-aware within country."
- `xtreg ..., fe` runs fixed effects (within transformation), with classical SEs by default. Add `vce(robust)` for HC1 or `vce(cluster cid)` for clustered SEs.

4. The do-file language

A do-file is a sequence of *commands*, one per line, with the following syntax skeleton:

```
[prefix:] command [varlist] [if exp] [in range] [weight] [, options]
```


4.1 Comments

Four comment forms are supported:

```
* Whole-line comment starting with asterisk
// Inline comment to end of line
/* Block comment
   spanning multiple lines */
generate x = 1 ///
      + 2      // line continuation: combine the next line with this one
```

4.2 Macros

Local macros are referenced as “``name’`” (back-tick ... tick); global macros as `$name`.

```
local x = 5
display "x = `x'"           // x = 5

global path "/home/me/data"
use "$path/employment.dta", clear

local vars "gdp pop unemp"
foreach v of local vars {
    summarize `v'
}
```

After most estimation commands, results are stored in the `e()` namespace:

```
regress y x1 x2
display e(N)           // sample size
display e(r2)          // R-squared
display _b[x1]         // x1's coefficient
display _se[x1]        // x1's standard error
```

Summarize stores results in `r()`:

```
summarize gdp
display r(mean)
display r(sd)
```

4.3 Control flow

```
* Conditional
if `n' > 100 {
    display "large sample"
}
else {
    display "small sample"
}

* Foreach over a list, varlist, or numlist
foreach v of varlist gdp pop unemp {
    quietly summarize `v'
```

```

    display "`v': N = " r(N) ", mean = " r(mean)
}

* Forvalues
forvalues i = 1/5 {
    display "iteration `i'"
}

```

4.4 Capture and quietly

`quietly` suppresses output; `capture` additionally suppresses errors (storing the return code in `_rc`):

```

quietly regress y x           // runs, no output
display _b[x]                 // coefficient is still available

capture confirm variable z // doesn't error if z doesn't exist
if _rc != 0 {
    display "z is missing"
}

```

4.5 The `by:` prefix

```

sort country year           // required before by:
by country: generate dy = y - y[_n-1]

bysort country (year): generate y_first = y[1] // sorts first

```

`tea` enforces the sort requirement — using `by:` on unsorted data is a hard error, not a silent miscomputation. Use `bysort` when you want it to sort automatically.

5. Data management

5.1 Importing data

`tea` reads CSV, TSV, Stata `.dta`, Excel `.xlsx`, and OpenDocument `.ods` files, plus its own native `.tea` binary format.

```

import delimited "data.csv", clear           // CSV
import delimited "data.tsv", clear           // TSV (auto-detected)
import excel "data.xlsx", firstrow clear
import excel "data.xlsx", sheet("Sheet2") firstrow clear

use "data.dta", clear                        // Stata format
use "data.tea", clear                        // native tea format

```

`tea` auto-detects the delimiter for `.csv` and `.tsv`; if the file extension is ambiguous, pass `delimiter(",")` or `delimiter(tab)` explicitly.

The `firstrow` option (for Excel) tells `tea` to use the first row as variable names. Without it, columns are named A, B, etc.

CSV import handles RFC 4180–style quoted fields, including embedded commas and newlines.

5.2 Saving data

```
save "out.dta", replace           // Stata format (default for .dta)
save "out.tea", replace          // native tea format
export delimited "out.csv", replace
export delimited "out.tsv", replace
```

Format choice. Use `.dta` if the data will be read by Stata. Use `.tea` if it's a temporary file between tea sessions (faster to read/write, no readstat overhead). Use `.csv` or `.tsv` if a non-statistical tool will consume the data.

5.3 Generating and modifying variables

```
generate gdp_log = log(gdp)
generate growth = (gdp - gdp[_n-1]) / gdp[_n-1]
replace growth = . if year == 1990           // missing for first year

generate str8 region = "WEST"
replace region = "EAST" if country_id == "CHN"
```

The `str#` prefix declares a string variable (the number is the maximum width, used by Stata when writing `.dta` files; in `tea` it's cosmetic on read but compressed back on write).

5.4 egen extended generators

`egen` provides aggregate functions that `generate` can't do in a single expression:

```
egen mean_gdp      = mean(gdp), by(country)
egen total_pop     = total(pop)
egen rank_gdp      = rank(gdp)
egen group_id      = group(country region)           // assigns 1..K
egen row_total     = rowtotal(x1 x2 x3 x4)
egen has_any       = anyvalue(x), values(1 5 9)
```

The full list of functions: `mean`, `sum`, `total`, `count`, `min`, `max`, `sd`, `var`, `median`, `group`, `rank`, `rowtotal`, `rowmean`, `rowmin`, `rowmax`, `rowmiss`, `rownonmiss`, `tag`, `seq`, `anyvalue`, `concat`.

5.5 Type conversion

```
* Numeric -> string
tostring year, generate(year_str)

* String -> numeric (errors if non-numeric values present)
destring score, generate(score_n)
destring score, generate(score_n) force           // converts non-numeric to .

* String -> integer codes + value label
encode country_id, generate(cid)                 // alphabetical 1..K

* Integer codes -> string (inverse of encode)
decode cid, generate(country_str)
```

```
* Recoding numeric values
recode age (0/17 = 1 "minor") (18/64 = 2 "working") (65/max = 3 "senior"), gen(age_cat)
```

5.6 Labels

```
label data "World Economic Outlook subset"
label variable growth "Real GDP growth (%)"

label define region 1 "North" 2 "South" 3 "East" 4 "West"
label values region region
```

6. Exploring data

6.1 describe

```
describe                // all variables
describe gdp pop        // just these two
describe gdp*           // wildcards work
```

Shows variable name, storage type, display format, and variable label.

6.2 list

```
list                    // all rows (capped at 200)
list in 1/10            // first 10 rows
list country year gdp if year == 2020
```

Column widths auto-fit to the maximum of header and value widths.

6.3 summarize

```
summarize               // all numeric variables
summarize gdp, detail   // adds percentiles, skewness, kurtosis
summarize gdp if year >= 2000
bysort country: summarize gdp
```

After `summarize`, the following `r()` macros are set: `r(N)`, `r(mean)`, `r(sd)`, `r(min)`, `r(max)`, `r(sum)`, `r(Var)`. With `detail`, also `r(p1)`, `r(p5)`, `r(p10)`, `r(p25)`, `r(p50)`, `r(p75)`, `r(p90)`, `r(p95)`, `r(p99)`, `r(skewness)`, `r(kurtosis)`.

6.4 tabulate

```
tabulate region          // one-way frequency table
tabulate region year, row column cell // two-way + percentages
tabulate region, summarize(gdp) means // means of gdp by region
```

6.5 codebook

```
codebook                // all variables
codebook gdp            // just one
```

Reports type, unique values, missing count, range, and distribution summary. More compact than `summarize`, `detail` for screening a new dataset.

6.6 count

```
count                  // total observations
count if missing(gdp)
count if gdp > 0 & year >= 2000
```

After `count`, `r(N)` holds the count.

7. Reshaping, merging, aggregating

7.1 sort and gsort

```
sort country year      // ascending on both
gsort -gdp             // descending on gdp
gsort country -year    // mixed
```

7.2 reshape

`reshape` is in-place — to keep the original you must `frame copy` first.

```
* Wide to long: each yY variable becomes y at year Y
reshape long y, i(country) j(year)

* Long to wide
reshape wide y, i(country) j(year)

* Multiple stubs
reshape long y z, i(country) j(year)
```

7.3 merge

```
merge 1:1 country year using "macro.dta"
merge m:1 country      using "country_attrs.dta"
merge 1:m country year using "panel_history.dta"
```

After `merge`, the variable `_merge` encodes the match status: 1 = master only, 2 = using only, 3 = matched.

`m:m` is deliberately rejected. If you think you need `m:m`, you almost certainly want a different operation (joinby-style cross product, or an explicit deduplication before merging).

7.4 collapse

```
collapse (mean) gdp pop (sum) revenue (sd) sd_gdp = gdp, by(country)
collapse (median) gdp [aweight=pop], by(region year)
```

Available aggregators: mean, median, sum, min, max, sd, count, first, last, p25, p50, p75, p10, p90.

7.5 Frames

A frame is a named in-memory dataset. Multiple frames let you keep several datasets loaded at once.

```
frame create macro
frame change macro
use "macro.dta", clear

frame change default
* now we're back on the original frame

frame copy default backup    // duplicate default into a new frame
frame drop backup
```

8. Time series and panel data

8.1 Declaring panel / time structure

```
tsset year                // pure time series
xtset country year        // panel
xtset country year, yearly // explicit time unit
```

After this, time-series operators are valid:

Operator	Meaning
L.x	first lag (one period back)
L2.x	second lag
L(1/3).x	first through third lags
F.x	first lead
D.x	first difference
D2.x	second difference (Stata's notation)
S.x	seasonal difference

These can mix with factor variables (with one limitation; see §9):

```
regress y L.y x i.region    // OLS with lag and region dummies
xtreg y L(1/3).y x, fe      // panel FE with three lags
```

tea's time-series operators are *gap-aware*: if a country has data for 1995 but not 1996, L.y for 1997 returns missing (not the 1995 value).

8.2 xtdescribe

```
xtdescribe
```

Reports number of panels, observations per panel (min/avg/max), and sample pattern. Useful for checking whether a panel is balanced.

9. Factor variables

Stata’s factor-variable grammar lets you include categorical regressors and their interactions inline, without manually generating dummies. **tea** supports the same grammar across every estimator (**regress**, **xtreg**, **logit**, **probit**, **poisson**, **ivregress**).

9.1 Atoms

Token	Meaning
<code>i.var</code>	dummies for each non-base level; base = smallest level
<code>ib2.var</code>	same, with base level explicitly set to 2
<code>c.var</code>	continuous regressor (mostly for use inside #)

```
regress wage exper i.region           // region dummies
regress wage exper ib2.region         // base = region 2
```

`i.var` requires `var` to be numeric. For string identifiers (e.g. ISO country codes), use **encode** first:

```
encode country_id, gen(cid)
regress y x i.cid
```

9.2 Interactions

Token	Meaning
<code>A#B</code>	cross product: $(K_A - 1) \times (K_B - 1)$ columns
<code>A##B</code>	main effects + interaction (equivalent to <code>A B A#B</code>)

```
regress wage i.region#c.exper         // region-specific slopes
regress wage i.region##c.exper        // adds main effects of region and exper
regress wage i.region#i.year          // region-by-year interaction
```

The `c.` prefix tells **tea** to treat the variable as continuous in an interaction — without it, a numeric variable would be expanded as a factor.

9.3 Coefficient naming

Factor expansions produce coefficients with structured names that **predict**, **margins**, and **_b[]** all understand:

```
regress wage i.region
display _b[2.region]           // coefficient on region 2 dummy
display _b[3.region#c.exper]    // interaction coefficient
```

9.4 Known limitation: factor $\times$ TS-op composition

Tokens like `i.country#c.L.gdp` (factor interaction with a lagged regressor) are **not** supported in v1.0.

Workaround: pre-compute the lag manually and use the bare variable inside the interaction:

```
generate L1_gdp = L.gdp
regress y i.country#c.L1_gdp           // works
```

10. Linear regression

10.1 Basic syntax

```
regress y x1 x2 x3
regress y x1 x2, vce(robust)           // HC1 robust SEs
regress y x1 x2, vce(cluster country) // CR1 clustered SEs
regress y x1 x2 [fweight=n], vce(robust) // frequency weights
```

Weight types: `fweight` (frequency, integer), `aweight` (analytic, rescaled to sum to N), `pweight` (probability, no rescaling), `iweight` (importance, like `aweight` but no rescaling).

10.2 Output

A standard ANOVA table with F -statistic and R^2 , followed by the coefficient table with t -statistics, p -values, and 95% confidence intervals. After `regress`, the following are stored:

- `e(N)`, `e(r2)`, `e(r2_a)`, `e(rmse)`, `e(F)`, `e(p)`, `e(df_r)`, `e(df_m)`
- `_b[varname]` and `_se[varname]` for each coefficient, plus `_b[_cons]` and `_se[_cons]`.

10.3 Robust and clustered SEs

The implementation uses HC1 for robust and CR1 for clustered, matching Stata's defaults. Cluster variables may be string (panel IDs) or numeric.

10.4 Constraints and hypothesis tests

After `regress`, use `test` for joint hypotheses and `lincom` for linear combinations:

```
regress y x1 x2 x3 x4

test x1 = x2           // single restriction
test x1 = 0, x2 = 0    // joint (F-test)
test x1 x2 x3          // all = 0 jointly

lincom x1 + x2          // linear combination + CI
lincom 0.5*x1 + 0.5*x2 - x3
```


11. Panel regression

11.1 Fixed effects

```
xtset country year
xtreg y x1 x2, fe // within (default for xtreg)
xtreg y x1 x2, fe vce(cluster country)
```

Output includes:

- Within / between / overall R^2
- σ_u, σ_e, ρ (variance components)
- F -test of $u_i = 0$
- `corr(u_i, Xb)` — informally measures the FE-RE departure

Identifies as `e(cmd) = "xtreg"` with `e(model) = "fe"`.

11.2 Random effects

```
xtreg y x1 x2, re // Swamy-Arora FGLS
```

Implementation: Swamy-Arora variant (the Stata default). Two-step FGLS — first estimate σ_u^2, σ_e^2 from the within and between residuals; then quasi-demean using $\theta = 1 - \sigma_e / \sqrt{T\sigma_u^2 + \sigma_e^2}$.

11.3 Between effects

```
xtreg y x1 x2, be // OLS on panel means
```

Collapses the data to one observation per panel (the panel means of y and each x), then runs OLS. This is the BE estimator that underpins one of the components of the RE-FE decomposition.

11.4 Hausman test

```
quietly xtreg y x1 x2, fe // populates fe_est
quietly xtreg y x1 x2, re // populates re_est
hausman // tests FE vs RE
```

Order of `xtreg fe` and `xtreg re` doesn't matter; `tea` keeps both in dedicated workspace slots and the `hausman` command (no arguments needed) tests their coefficient difference.

12. Maximum likelihood estimators

`tea` implements logit, probit, and Poisson regression via a shared Newton-Raphson driver. All three support `if`, `in`, `weights`, and `vce(robust|cluster)`.

12.1 logit

```
logit voted income education
logit y x1 x2 i.region, vce(cluster country)
```

Reports the maximum likelihood iteration trace, LR χ^2 test against the constant-only model, pseudo R^2 , and the coefficient table with z -statistics.

12.2 probit

Same syntax, normal CDF instead of logistic CDF:

```
probit y x1 x2 i.region
```

12.3 poisson

For count outcomes:

```
poisson accidents speed_limit population, vce(cluster state)
```

The link is $\log E[y \mid x] = x'\beta$; coefficients are log-rate-ratios. $y < 0$ is rejected. Non-integer y is permitted (treated as the GLM extension).

Important: `tea`'s Poisson drops the $\log(y!)$ constant in the log-likelihood, so the absolute `e(11)` value differs from Stata's, but the coefficient estimates, SEs, and likelihood-ratio tests are identical.

12.4 Marginal effects via margins

After any of the above, `margins` produces average or at-the-mean marginal effects:

```
logit y x1 x2 i.region
margins, dydx(*)           // average marginal effects
margins, dydx(x1)         // just x1
margins, dydx(*) atmeans  // at-the-means MEs
```

Standard errors use the delta method.

13. Instrumental variables

`ivregress 2sls` implements two-stage least squares with a first-stage F diagnostic for weak instruments.

13.1 Syntax

```
ivregress 2sls y [exog_vars] (endog_vars = instruments) [if] [in] [weight] [, options]
```

The parenthesized group is the key piece: variables to the left of the `=` are endogenous; variables to the right are excluded instruments.

13.2 Example

```
* Wage equation: education is endogenous, parents' education is the IV
ivregress 2sls wage exper exper2 (educ = momeduc dadeduc), vce(robust)
```

Output includes the coefficient table (with z -statistics from asymptotic normality), the list of instrumented variables, the list of instruments (included exogenous + excluded), and the first-stage F -statistic for the weakest endogenous regressor.

13.3 Interpreting the first-stage F

A common rule of thumb (Stock-Yogo): if the first-stage F is below 10, the instrument is weak and the IV estimates may have substantial finite-sample bias. **tea** flags this in the output with a caret.

13.4 Variance estimators

```
ivregress 2sls y (x = z) // classical
ivregress 2sls y (x = z), vce(robust) // HC1
ivregress 2sls y (x = z), vce(cluster country) // CR1
```

The sandwich form is $V = A \cdot B \cdot A$ where $A = (\hat{X}'\hat{X})^{-1}$ (using fitted endogenous columns) and B uses residuals computed from the *original* X (this is the correct 2SLS variance — not the naive OLS-on-fitted variance).

14. Time series models

14.1 ARIMA

```
tsset year
arima y, arima(p d q) [noconstant]
arima y x1 x2, arima(1 0 1) // ARMAX with two exog
```

Specifies an ARIMA(p, d, q) model: AR order p , difference order d , MA order q . Exogenous regressors can be included (then the model is ARMAX).

Important. v1.0's **arima** uses *conditional likelihood*, not the Kalman-filter exact ML that Stata uses by default. The differences:

- Coefficient estimates are consistent and converge to the same limit, but finite-sample values can differ by a few percent.
- The absolute log-likelihood differs (we drop the initial-state contribution).
- Standard errors come from a numerical-difference Hessian and can be off by 1–5% on the MA components specifically. Point estimates are unaffected.

If you need exact ML, escape to R's **arima()** or Python's **statsmodels.tsa.arima**.

14.2 Example

```
* US growth ARIMA(1,0,0)
tsset year
arima growth, arima(1 0 0)
```

The output groups coefficients into “ARMA model” (constant + exog), “AR” (ϕ_i), “MA” (θ_j), and /sigma (the innovation standard deviation).

15. Post-estimation

15.1 predict

After any estimator, `predict` generates a new variable with the model prediction or a residual:

```
* After regress
regress y x1 x2
predict yhat                // linear prediction (default: xb)
predict resid, residuals    // residuals
predict resid, r            // same, shorter

* After logit/probit
logit voted income
predict p                   // Pr(y=1)
predict xb, xb              // linear index

* After xtreg fe
xtreg y x1, fe
predict u, u                // estimated u_i
predict e, e                // idiosyncratic residual
predict ue, ue              // u_i + e_it
```

15.2 Hypothesis tests

`test` performs Wald tests on linear restrictions:

```
regress y x1 x2 x3
test x1 = 0                // single restriction
test x1 = 1, x2 = 0        // joint restriction
test x1 x2 x3              // x1 = x2 = x3 = 0
```

`lincom` computes linear combinations with their SEs and 95% CIs:

```
lincom x1 + x2 - x3
lincom 0.5*x1 + 0.5*x2
```

16. Output and reproducibility

16.1 Saving estimates

The `estimates` (alias `est`) command suite saves and manages named estimation results:

```

regress y x1
estimates store m1

regress y x1 x2
estimates store m2

regress y x1 x2 x3
estimates store m3

estimates dir                      // list stored estimates
estimates table m1 m2 m3, stats(N r2 rmse) star
estimates restore m1              // load m1 back to last_est
estimates drop m2                 // remove m2
estimates drop _all               // remove all

```

16.2 estout: publication tables

For papers, use `estout` to produce LaTeX, markdown, or plain side-by-side tables:

```

estout m1 m2 m3                      // default: LaTeX
estout m1 m2 m3, format(latex) stats(N r2 rmse)
estout m1 m2 m3, format(markdown) se star
estout m1 m2 m3, format(plain) t

estout m1 m2 m3 using "table.tex", stats(N r2) // write to file
estout m1 m2 m3, title("Regression results") label(tab:main)

* Subset of coefficients
estout m1 m2, keep(x1 x2 _cons) stats(N r2)
estout m1 m2, drop(_cons)           // drop constant row

```

Options:

Option	Effect
<code>format(latex markdown plain)</code>	output format (default: <code>latex</code>)
<code>se, t, p</code>	show SEs / <i>t</i> -stats / <i>p</i> -values below coefs
<code>star</code>	add significance stars (10%, 5%, 1%)
<code>stats(N r2 r2_a rmse F ...)</code>	footer rows
<code>title(...), label(...)</code>	LaTeX caption + reference label
<code>keep(...), drop(...)</code>	restrict coefficient rows
<code>nomtitles, mtitles(...)</code>	override per-model column labels
<code>using FILE</code>	write to file instead of stdout

LaTeX output produces a stand-alone `tabular` block you can `\input` in your paper.

16.3 log

```

log using "session.log", replace
* commands and their output are captured...
log close

```

Captures both the commands you ran (echoed with a `.` prefix) and the output they produced.

17. Random numbers and Monte Carlo

17.1 Reproducibility

```
set seed 12345                                // fix the PRNG state
```

`tea` uses a 64-bit PCG generator. `set seed N` is deterministic — the same seed always produces the same sequence, across runs, machines, and `tea` versions.

17.2 Functions

```
generate u = runiform()                        // Uniform[0, 1)
generate z = rnormal()                        // Standard normal
generate w = rnormal(5, 2)                    // Normal(mean=5, sd=2)
```

Normal draws use Box-Muller with caching. All three functions are available in any expression context (`generate`, `replace`, `egen rowfirst/rowlast`, etc.).

17.3 A Monte Carlo example

```
* OLS slope sampling distribution under known DGP
set seed 42
clear
set obs 1000

* True model: y = 2*x + eps, eps ~ N(0, 1)
generate x = rnormal()
generate eps = rnormal()
generate y = 2*x + eps

regress y x
display "Estimated beta = " _b[x] " (true is 2.0)"
```

18. Escape hatches: when to leave tea

`tea` is excellent at the data-prep-to-quick-estimate loop, but many things are out of scope. The escape pattern is:

1. Do data prep in `tea`.
2. `export delimited` to CSV.
3. `shell python myscript.py` (or R, or anything).
4. `import delimited` the results back.

18.1 When to escape

What you need	Where to go
Plotting	gnuplot, Python's matplotlib, R's ggplot2
GARCH / EGARCH / TGARCH	Python arch, R rugarch
Exact-ML ARIMA (Kalman)	R's arima(), forecast::Arima
Seasonal ARIMA	R's arima() with seasonal arg
Bayesian inference	PyMC, Stan (rstan, cmdstanpy)
Mixed-effects models	R's lme4, Python statsmodels.MixedLM
Survival analysis	R's survival, Python lifelines
Cointegration / VAR / VECM	R's urca, vars
Machine learning	scikit-learn, xgboost, lightgbm
Network / graph models	networkx, R igraph
Spatial econometrics	R's spdep, Python's pysal

18.2 A bridge example

```
* Estimate panel FE in tea, then use Python for a residuals plot
xtset country year
xtreg gdp_growth L.gdp_growth inflation, fe
predict resid, e

* Export for plotting
keep country year resid
export delimited "resid.csv", replace

* Hand off to Python (assuming plot_residuals.py exists)
shell python plot_residuals.py resid.csv resid_plot.png
```

19. Known v1.0 limitations

These are not bugs in the strict sense — they're scope decisions for v1.0 that we may revisit based on testing.

19.1 Data and metadata

- **.dta round-trip does not preserve xtset state.** After `save mydata.dta` then use `mydata.dta`, the panel/time variable assignment is lost. Workaround: re-run `xtset country year` after use. This is the standard Stata-do-file convention anyway.
- **Variable name length cap of 32 characters.** Factor-variable interaction names like `2010.year#1985.country` (28 chars) fit; longer 3-way interactions truncate.

19.2 Econometric estimators

- **ARIMA uses conditional likelihood, not Kalman-filter exact ML.** See §14 for details. For serious work, use R's `arima()`.
- **No seasonal ARIMA** (`sar`, `sma` terms).

- **ARIMA MA standard errors** have a 1–5% precision loss from the numerical-Hessian computation. Point estimates are unaffected.
- **xtreg, be** uses **straight OLS** on panel means. Stata’s default applies a slight variance correction for unbalanced panels (down-weighting groups with few observations). Differences appear only on unbalanced panels and only in the SEs (point estimates are identical).
- **No xtabond / system GMM** or other GMM estimators.
- **No VAR / VECM**. Escape to R.

19.3 Factor variables

- **No composition with TS operators**. Tokens like `i.country#c.L.gdp` are not parsed. Workaround: pre-compute the lag with `gen L1_gdp = L.gdp` and use `c.L1_gdp`.
- **i.var requires numeric var**. Use `encode` on string identifiers.
- **Cell-count cap of 10 000**. Larger factor expansions are blocked.

19.4 Programming

- **No program define, no syntax parser, no Mata**. You can write loops and use macros, but you cannot define new commands.

19.5 Graphics

No graphics commands. Use `export delimited` and pipe to `gnuplot` or to Python / R.

20. Reporting bugs

If you find a bug:

1. Distill it into a small reproducer — a do-file plus a small input file is ideal. If the input data is sensitive, replace it with `set seed N; gen x = rnormal()` or similar synthetic.
2. Note your `tea --version`, OS, and compiler version.
3. Send to the author at the email associated with your distribution of `tea`. If you don’t have one, ask the person who gave you the software.

What counts as a bug:

- Wrong numerical answer (with hand-calculated expected value).
- Crash, segfault, or hang on input that should work.
- Different behavior from documented Stata semantics (when not in the “known limitations” list above).

What does not count as a bug:

- Missing feature (those go on the v1.1 feature-request list).
- Disagreement with Stata’s exact numerical output to many decimal places (small differences arise from BLAS vendor variation and are not correctness issues).

- Things listed in §20 above as known limitations.

A. Command reference

A one-line summary of every command in v1.0. For full syntax, use `help CMD` in the REPL.

Data I/O

- `import delimited FILE [, clear delimiter(x) varnames(N)]`
- `import excel FILE [, sheet(name) firstrow clear]`
- `export delimited FILE [, replace delimiter(x)]`
- `use FILE [, clear]`
- `save FILE [, replace]`
- `clear`

Variable creation and modification

- `generate [type] varname = exp [if] [in]`
- `replace varname = exp [if] [in]`
- `egen varname = func(...) [, by(varlist)]`
- `drop varlist | if exp | in range`
- `keep varlist | if exp | in range`
- `rename old new`
- `order varlist`
- `recode varname (rule) (rule) ... [, gen(newvar)]`
- `encode strvar, gen(newvar) [label(name)]`
- `decode encvar, gen(newstrvar)`
- `destring strvar, gen(newvar) [force]`
- `tostring numvar, gen(newvar)`
- `format varlist %fmt`
- `label variable varname "label"`
- `label define name # "label" ...`
- `label values varname name`
- `label list`

Exploration

- `describe [varlist]`
- `list [varlist] [if] [in]`
- `summarize [varlist] [if] [in] [, detail]`
- `tabstat varlist [, by(g) statistics(...)]`
- `tabulate var1 [var2] [, row column cell missing]`
- `codebook [varlist]`
- `count [if]`

Sorting, reshaping, merging

- `sort varlist`
- `gsort [+|-]var [+|-]var ...`
- `reshape long stubs, i(idvars) j(jvar)`
- `reshape wide stubs, i(idvars) j(jvar)`
- `merge 1:1 | m:1 | 1:m keyvars using FILE [, options]`
- `collapse (stat) varlist [, by(g) weight]`
- `frame create|change|copy|drop ...`

Time series and panel

- `tsset timevar`
- `xtset panelvar timevar`
- `xtdescribe`

Estimators

- `regress depvar indepvars [if] [in] [weight] [, vce(...)]`
- `xtreg depvar indepvars [, fe|re|be vce(...)]`
- `hausman (after fe + re)`
- `logit depvar indepvars [, vce(...)]`
- `probit depvar indepvars [, vce(...)]`
- `poisson depvar indepvars [, vce(...)]`
- `ivregress 2sls depvar [exog] (endog = instruments) [, vce(...)]`
- `arma depvar [exog], arima(p d q) [noconstant]`

Post-estimation

- `predict newvar [, xb|residuals|u|e|ue|p|xb]`
- `margins, dydx(*|varlist) [atmeans]`
- `test exp [, accumulate]`
- `lincom exp`

Output

- `estimates store NAME`
- `estimates restore NAME`
- `estimates dir`
- `estimates drop NAME | _all`
- `estimates table NAMES [, se|t|p star stats(...)]`
- `estout NAMES [using FILE] [, format(...) se|t|p star stats(...) keep(...) drop(...) title(...) label(...) mtitles(...) nomtitles]`
- `log using FILE [, replace append]`
- `log close`

Programming

- `local name = exp or local name "string"`
- `global name = exp or global name "string"`
- `display expr`
- `foreach v of varlist|local|global|numlist ... { ... }`
- `forvalues i = start/end { ... }`
- `if exp { ... } [else { ... }]`
- `capture command`
- `quietly command`
- `set seed N`
- `set obs N`

System

- `help [CMD]`
- `pwd, cd PATH, mkdir DIR, rmdir DIR`
- `dir [pattern] (or ls), erase FILE (or rm), copy SRC DST`
- `do FILE`

- `shell CMD` (or `!CMD`)
- `version`, `exit`

B. Function reference

Functions available in any expression context (`generate`, `replace`, `if` clauses, `display`, ...). Each entry shows the signature and a one-line description. In all cases: a missing argument propagates to a missing result, unless otherwise stated.

B.1 Arithmetic and rounding

Signature	Returns
<code>abs(x)</code>	$ x $
<code>int(x)</code>	truncate toward zero
<code>floor(x)</code>	largest integer $\leq x$
<code>ceil(x)</code>	smallest integer $\geq x$
<code>round(x)</code>	round to nearest integer (half away from zero)
<code>round(x, unit)</code>	round to nearest multiple of <code>unit</code>
<code>sign(x)</code>	$-1 / 0 / +1$
<code>mod(x, y)</code>	$x \bmod y$, same sign as x
<code>min(x, y, ...)</code>	minimum of arguments (missing skipped)
<code>max(x, y, ...)</code>	maximum of arguments (missing skipped)
<code>cond(test, a, b)</code>	a if <code>test</code> is true, else b

B.2 Exponential and logarithm

Signature	Returns
<code>exp(x)</code>	e^x
<code>ln(x)</code>	natural log; missing if $x \leq 0$
<code>log(x)</code>	alias for <code>ln</code>
<code>log10(x)</code>	base-10 log; missing if $x \leq 0$
<code>sqrt(x)</code>	$\sqrt{x}$; missing if $x < 0$

B.3 Trigonometric and hyperbolic

Signature	Returns
<code>sin(x)</code> , <code>cos(x)</code> , <code>tan(x)</code>	radians
<code>asin(x)</code> , <code>acos(x)</code>	inverse trig; range checked
<code>atan(x)</code>	inverse tangent
<code>atan2(y, x)</code>	two-argument inverse tangent
<code>sinh(x)</code> , <code>cosh(x)</code> , <code>tanh(x)</code>	hyperbolic

B.4 Probability distributions

Signature	Returns
<code>normal(z)</code>	standard-normal CDF $\Phi(z)$
<code>normalden(z)</code>	standard-normal PDF $\phi(z)$
<code>normalden(z, mu, sd)</code>	normal PDF with mean and SD
<code>lnnormal(z)</code>	$\log \Phi(z)$, stable in the lower tail
<code>lnnormalden(z)</code>	$\log \phi(z)$, stable in extremes
<code>invnormal(p)</code>	inverse CDF $\Phi^{-1}(p)$
<code>ttail(df, t)</code>	upper-tail Student- t : $P(T > t)$
<code>invttail(df, p)</code>	inverse: t such that $P(T > t) = p$
<code>F(df1, df2, F)</code>	lower-tail F CDF
<code>Ftail(df1, df2, F)</code>	upper-tail F : $P(F > f)$
<code>chi2(df, x)</code>	lower-tail χ^2 CDF
<code>chi2tail(df, x)</code>	upper-tail χ^2 : $P(X > x)$

B.5 Random number generation

Reproducible from `set seed N`. See §17.

Signature	Returns
<code>runiform()</code>	uniform $[0, 1)$
<code>rnormal()</code>	standard normal
<code>rnormal(mu)</code>	normal with mean <code>mu</code> , sd 1
<code>rnormal(mu, sd)</code>	normal with mean <code>mu</code> and SD <code>sd</code>

B.6 Missing-value tests and ranges

Signature	Returns
<code>missing(x)</code>	1 if <code>x</code> is any missing code, else 0
<code>inrange(x, lo, hi)</code>	1 if $lo \leq x \leq hi$
<code>inlist(x, v1, v2, ...)</code>	1 if <code>x</code> equals any of <code>v1</code> , <code>v2</code> , ...

B.7 String construction and inspection

Signature	Returns
<code>strlen(s), length(s)</code>	character count of <code>s</code>
<code>substr(s, start, len)</code>	substring; <code>start</code> is 1-based; -1 for last
<code>strpos(s, sub)</code>	1-based index of <code>sub</code> in <code>s</code> , 0 if absent
<code>strrpos(s, sub)</code>	like <code>strpos</code> but search from the right
<code>word(s, n)</code>	the n th whitespace-separated word
<code>wordcount(s)</code>	number of whitespace-separated words
<code>abbrev(s, n)</code>	abbreviate <code>s</code> to width <code>n</code>
<code>char(n)</code>	single-character string for ASCII code <code>n</code>

B.8 String transformation

Signature	Returns
<code>strupper(s), upper(s)</code>	uppercase
<code>strlower(s), lower(s)</code>	lowercase
<code>strproper(s), proper(s)</code>	title case (first letter of each word)
<code>strtrim(s), trim(s)</code>	strip leading and trailing whitespace
<code>ltrim(s), rtrim(s)</code>	strip leading / trailing whitespace
<code>itrim(s)</code>	collapse internal whitespace runs to one space
<code>strreverse(s)</code>	reverse character order
<code>subinstr(s, from, to)</code>	replace all occurrences of <code>from</code> with <code>to</code>
<code>subinstr(s, from, to, n)</code>	replace at most <code>n</code> occurrences
<code>subinword(s, from, to)</code>	replace whole-word matches only
<code>strmatch(s, pattern)</code>	glob-style match (<code>*</code> , <code>?</code>); 1 / 0

B.9 Regular expressions (POSIX extended)

Signature	Returns
<code>regexpm(s, pat)</code>	1 if <code>pat</code> matches anywhere in <code>s</code> , else 0; populates <code>regexprs()</code>
<code>regexprs(n)</code>	<code>n</code> th captured group from last <code>regexpm</code> (0 = full match)
<code>regexpr(s, pat, repl)</code>	replace first match in <code>s</code> ; <code>repl</code> may use <code>\1 ... \9</code>

B.10 Numeric ↔ string conversion

Signature	Returns
<code>string(x)</code>	default decimal string
<code>string(x, fmt)</code>	format <code>x</code> with the given Stata format
<code>stroofreal(x), strofreal(x, fmt)</code>	alias for <code>string</code>
<code>real(s)</code>	parse <code>s</code> as numeric; missing on failure

B.11 Calendar: building dates

Signature	Returns
<code>mdy(month, day, year)</code>	daily date (<code>%td</code>) for (month, day, year)
<code>ym(year, month)</code>	monthly date (<code>%tm</code>)
<code>yq(year, quarter)</code>	quarterly date (<code>%tq</code>)
<code>yh(year, half)</code>	half-yearly date (<code>%th</code>)
<code>yw(year, week)</code>	weekly date (<code>%tw</code>)
<code>date(s, fmt)</code>	parse string <code>s</code> per <code>fmt</code> ("YMD", "DMY", "MDY", etc.)

B.12 Calendar: extracting components from a daily date

Signature	Returns
<code>year(d)</code>	calendar year
<code>month(d)</code>	month, 1–12
<code>day(d)</code>	day of month, 1–31
<code>quarter(d)</code>	quarter, 1–4
<code>halfyear(d)</code>	half-year, 1 or 2
<code>week(d)</code>	ISO week number
<code>dow(d)</code>	day of week, 0 (Sun) – 6 (Sat)
<code>doy(d)</code>	day of year, 1–366

B.13 Calendar: converting between frequencies

Signature	Returns
<code>mofd(d)</code>	monthly date corresponding to daily date <code>d</code>
<code>dofm(m)</code>	first day of month <code>m</code> (as daily)
<code>qofd(d)</code>	quarterly index of daily <code>d</code>
<code>dofq(q)</code>	first day of quarter <code>q</code>
<code>hofd(d)</code>	half-year index of daily <code>d</code>
<code>dofh(h)</code>	first day of half-year <code>h</code>
<code>wofd(d)</code>	weekly index of daily <code>d</code>
<code>ty(y)</code>	year-as-yearly-date (just <code>y</code> , for display)
<code>dofy(y)</code>	first day of year <code>y</code>
<code>tm, tq, tw, th, td</code>	identity converters used for format-tagging

B.14 Special: running and time-series-context

Signature	Returns
<code>sum(x)</code>	running cumulative sum (resets per by-group)
<code>_n</code>	current observation index (1-based)
<code>_N</code>	total observation count in current scope
<code>x[_n-1], x[_n+1]</code>	lag / lead within current sort order
<code>x[1], x[_N]</code>	first / last observation of current scope

C. egen function reference

Functions usable as `egen newvar = func(...)`. See §5.4.

C.1 Group-wise aggregators

When combined with `by(grouplist)`, these compute one value per group and broadcast that value back to every row in the group. Without `by()`, they collapse to a single scalar broadcast everywhere.

Signature	Returns
<code>mean(exp)</code>	arithmetic mean
<code>sum(exp), total(exp)</code>	sum (missing values skipped)
<code>count(exp)</code>	number of non-missing observations
<code>min(exp), max(exp)</code>	extrema
<code>sd(exp)</code>	sample standard deviation, $n - 1$ denominator
<code>var(exp)</code>	sample variance
<code>median(exp)</code>	50th percentile (linear interpolation)
<code>iqr(exp)</code>	interquartile range, 75th – 25th
<code>pc(exp)</code>	percentile; option <code>p(N)</code> sets which (default 50)

C.2 Position and identity

Signature	Returns
<code>rank(exp)</code>	1-based rank within group; ties handled by mean rank
<code>group(varlist)</code>	1...K integer ID per unique combination of <code>varlist</code>
<code>tag(varlist)</code>	1 on the first row of each <code>varlist</code> group, 0 otherwise

C.3 Row-wise aggregators (across columns of a single row)

These ignore `by()` (the row is the unit).

Signature	Returns
<code>rowtotal(varlist)</code>	sum across columns; missing skipped
<code>rowmean(varlist)</code>	mean across columns
<code>rowmin(varlist)</code>	minimum across columns
<code>rowmax(varlist)</code>	maximum across columns
<code>rowsd(varlist)</code>	sample SD across columns
<code>rowmiss(varlist)</code>	count of missing cells in the row
<code>rownonmiss(varlist)</code>	count of non-missing cells

D. Summarize / tabstat statistics keywords

Available with `tabstat varlist, statistics(...)`, and many also as percentile selectors after `summarize, detail`.

Keyword	Meaning
<code>n</code>	non-missing count
<code>mean</code>	arithmetic mean
<code>sum</code>	total
<code>sd</code>	standard deviation (sample, $n - 1$)
<code>variance</code>	variance
<code>min, max</code>	extrema
<code>range</code>	<code>max - min</code>
<code>median</code>	50th percentile
<code>p1, p5, p10, p25, p50</code>	percentiles
<code>p75, p90, p95, p99</code>	percentiles
<code>iqr</code>	75th – 25th percentile
<code>skewness, kurtosis</code>	after <code>summarize</code> , <code>detail</code>

E. Stata cheat sheet

A quick reference for Stata users adapting to `tea`.

Same as Stata

- Syntax: `[prefix:] command [varlist] [if] [in] [weight] [, options]`
- Missing-value algebra, including `if x > 5` including missing
- Macros: `'local'` and `$global`, `r()` and `e()` returns
- `by:`, `bysort:`, the `[_n-1]` subscript, `_N`
- Value labels: `label define / values / list`
- Factor variables (`i.`, `c.`, `ib<n>.`, `#`, `##`)
- Time-series operators (`L.` `F.` `D.` `S.`)

Tea-specific differences

- `m:m` merge is rejected with an explanatory message.
- `xtreg`, `be` uses simple OLS on panel means.
- `arma` uses conditional likelihood.
- `i.country#c.L.gdp` doesn't parse; pre-compute the lag.
- No `.gph` graphics — use `export` and external tools.
- `program` `define`, `syntax`, and `Mata` are absent.

Stata commands that don't exist in tea

- Graphics: `graph`, `twoway`, `histogram`, etc.
- GMM: `gmm`, `xtabond`, `xtdpd`.

- Time series: `var`, `vec`, `dfgl`, `dfuller`, `xcorr`, `wntestq`.
- Survival: `stcox`, `streg`, `stset`.
- Mixed: `mixed`, `xtmixed`, `meqrlogit`.
- Bayesian: `bayes:`, `bayesmh`.
- Multivariate: `mvreg`, `factor`, `pca`.
- Multiple imputation: `mi`.

For each of these, see §18 for the recommended external tool.